

**Wishbone DMA/Bridge IP Core
Architecture document**

by

Subha Amancherla

Table of contents

1 Introduction.....	3
2 Architecture.....	3
3 Functional description.....	4
3.1 Channel select.....	4
3.1.1 Priority encoder.....	5
3.1.2 Channel arbiter.....	7
3.1.3 Multiplexer.....	10
3.2 Core registers.....	11
3.3 DMA Engine.....	12
3.4 Wishbone interface.....	15
3.4.1 Wishbone master.....	16
3.4.2 Wishbone slave.....	16
3.4.3 Handshaking Protocol.....	16
3.4.4 Single READ / WRITE Cycles.....	17
4 Parameters to be measured.....	17
5 Appendix.....	17

1 Introduction

The WISHBONE System-on-Chip (SoC) architecture is a portable interface for use with semiconductor IP cores. It's intended to be used as an internal bus for SoC applications. Its purpose is to foster design reuse by alleviating System-on-Chip integration problems. This is accomplished by creating a common interface between IP cores. This improves the portability and reliability of the system, and results in faster time-to-market for the end user. WISHBONE uses MASTER/SLAVE architecture. That means that functional modules with MASTER interfaces initiate data transactions to participating SLAVE interfaces. The MASTER and SLAVE communicate through an interconnection interface called the INTERCON. The INTERCON contains circuits. These circuits allow MASTER to communicate with SLAVE.

WISHBONE DMA/bridge core provides DMA transfers between two WISHBONE Interfaces. Transfers can also be performed on the same Wishbone interface. It can also act as a Bridge, allowing masters on each WISHBONE interface to directly access slaves on the other interfaces.

Reference

Reference to this document is http://www.opencores.org/projects/wb_dma/
rev1.5 dated: Jan 27,2002 Author : Rudolf usselmann

Objective

The objective of this design is to code the module in synthesis-able system verilog and then compare the performance and code compactness with the reference design, which is in verilog.

2 Architecture

The core consists of two Wishbone interfaces, DMA Engine, channel select, core registers and pass through logic as main modules.

Wishbone interface is Wishbone Soc bus specification Rev B compliant. The bus is 32 bits wide.

DMA Engine has upto 32 channels. It supports transfers on two interfaces as well as same interface. Channels can be programmed with different priorities. Channels with same priority are serviced in a round robin way.

Pass through logic performs bridging operation between two interfaces.

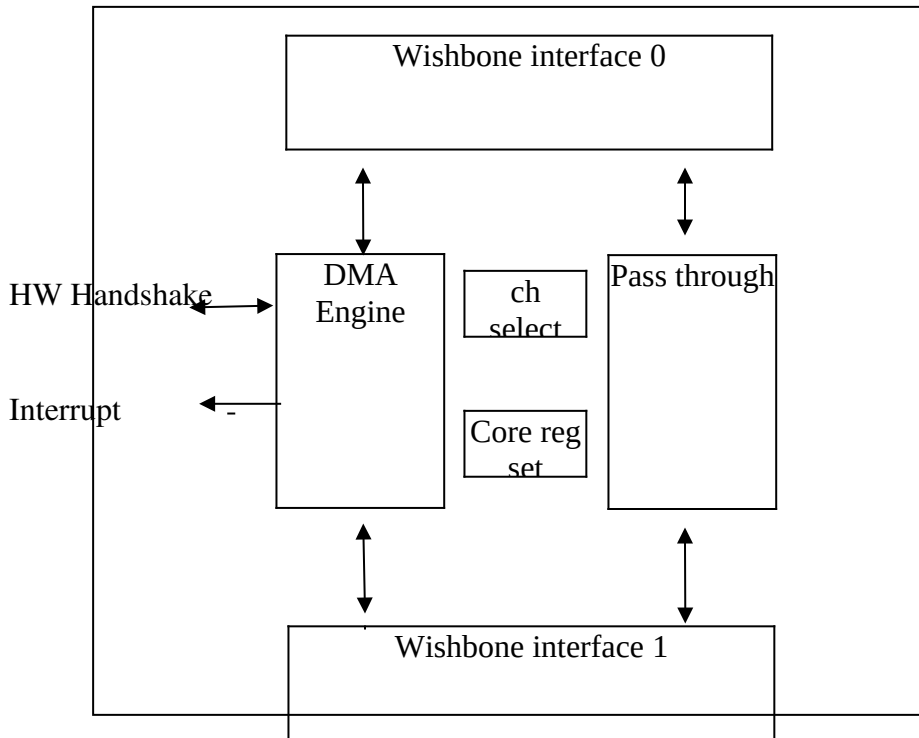


Fig 1: Basic architecture

3 Functional description

The core consists of DMA Engine, Wishbone interfaces, Channel select and core registers as major modules. DMA Engine module consists of normal DMA operations, HW handshake mode of operation, linked list descriptors and pass through operations. Each Wishbone interface consists of a master and a slave interface. Channel select consists of priority encoder, round robin channel arbiter and multiplexer. Core Registers contain output data multiplexer, address decoder and field registers.

3.1 Channel select

Channel select main functionality is to assert grant to one channel out of 32 channels. It interfaces with channels, core registers and also DMA engine. When request is there for DMA transfer, the channel select encodes the priority and through an arbiter decides to give a grant to particular channel and assert select lines onto multiplexer. The DMA engine will perform the DMA as per the request.

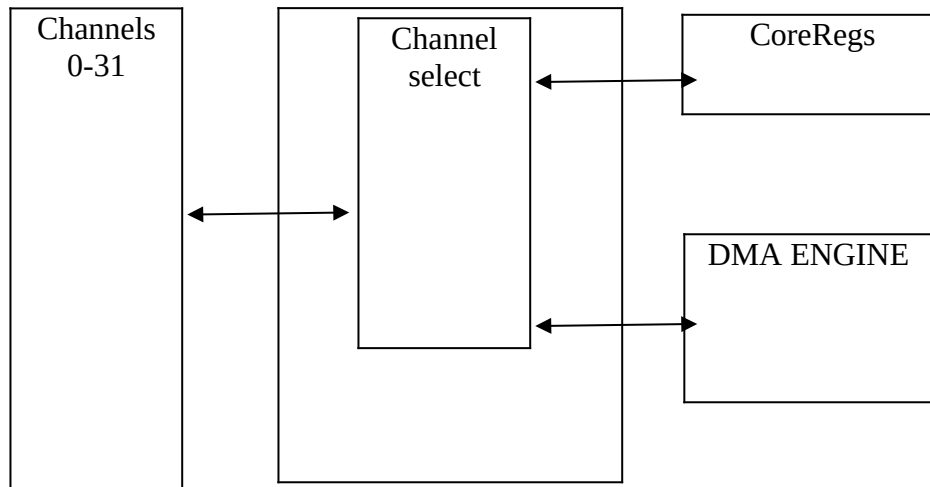


Fig 3. Showing the internal modules in channel select

The priority information comes from the register file. The request will trigger the priority encoder to encode the priority into 8 priority fifos. The channel arbiter will then get the trigger indicating the valid requests from the channels. Using the priority encoding the arbiter will decide to give grant to high-prioritized channel. If the request is from different channels with same priority then, the grant is served as per the round robin fashion. The multiplexer will then get the select signal and the granted signal outputs are placed on DMA interface.

3.1.1 Priority encoder

Priority encoder is designed to sort the channels as per their priority. When there is a request from host, the priority encoder gets trigger. It then, checks the validity of that channel using valid bit of that particular channel. Once the channel is valid, the channel number is written into specific fifo depending on its priority value. If the channel0 is assigned a priority of 6, the channel number 0 is written in priority6 fifo. The write pointer of prio_fifo6 is incremented. The combinational logic will set the pending request on that priority. In the similar fashion all valid channels will be written into their defined

priority fifos. There are 8 such fifos. The depth of each fifo is 32 bit and width is 3 bit wide. The empty signal on the fifos will remove the pending request on that fifo.

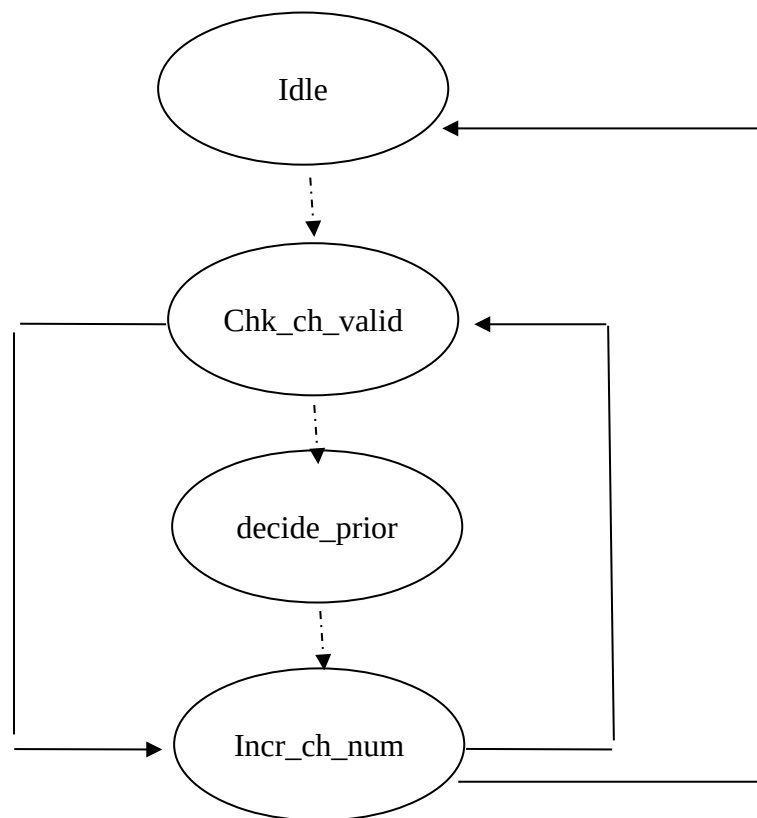


Fig 5: State machine for priority encoder

State description

Idle:

On reset the state is reached. When there is request from host, the state transition begins.

Idle -> chk_ch_valid : if request from software and assert ch_num = 0

Chk_ch_valid:

Checking of channel validity is performed here.

chk_ch_valid -> decide_prior : if ch_valid[ch_num] == 1

chk_ch_valid -> incr_ch_num : if ch_valid[ch_num] == 0

decide prior:

Depending on the priority value asserted for that channel the channel number is written into that particular fifo.

decide_prior -> incr_ch_num : After adding the channel number in desired priority fifo.

incr_ch_num:

The channel number is incremented. Also the check if all the channel numbers are verified or not is also done here. While going back to idle the arbiter begin should be asserted.

incr_ch_num -> chk_ch_valid : if ch_num < 32

incr_ch_num -> idle : if ch_num == 32

3.1.2 Channel arbiter

Once the Priority encoder finishes the sorting, all the priority fifos will display the pending requests. Arbiter begin signal gets asserted from priority arbiter. The arbiter has to pump out the channel selects onto multiplexer reading the priority fifos from highest priority. The arbiter has to poll for the next channel signal from DMA. Once the next channel is asserted the arbiter should check for priority level that is being implemented. If the level is 8 then, the arbiter should begin checking the priority from prior_fifo7. If the priority level is 4 then it should start from checking from prior_fifo4. Else if the priority level is 2 then it can start checking from prior_fifo2 and continue reading the prior_fifos below it. Once all fifos are read, the arbiter goes back to higher priority fifo to serve the pending request.

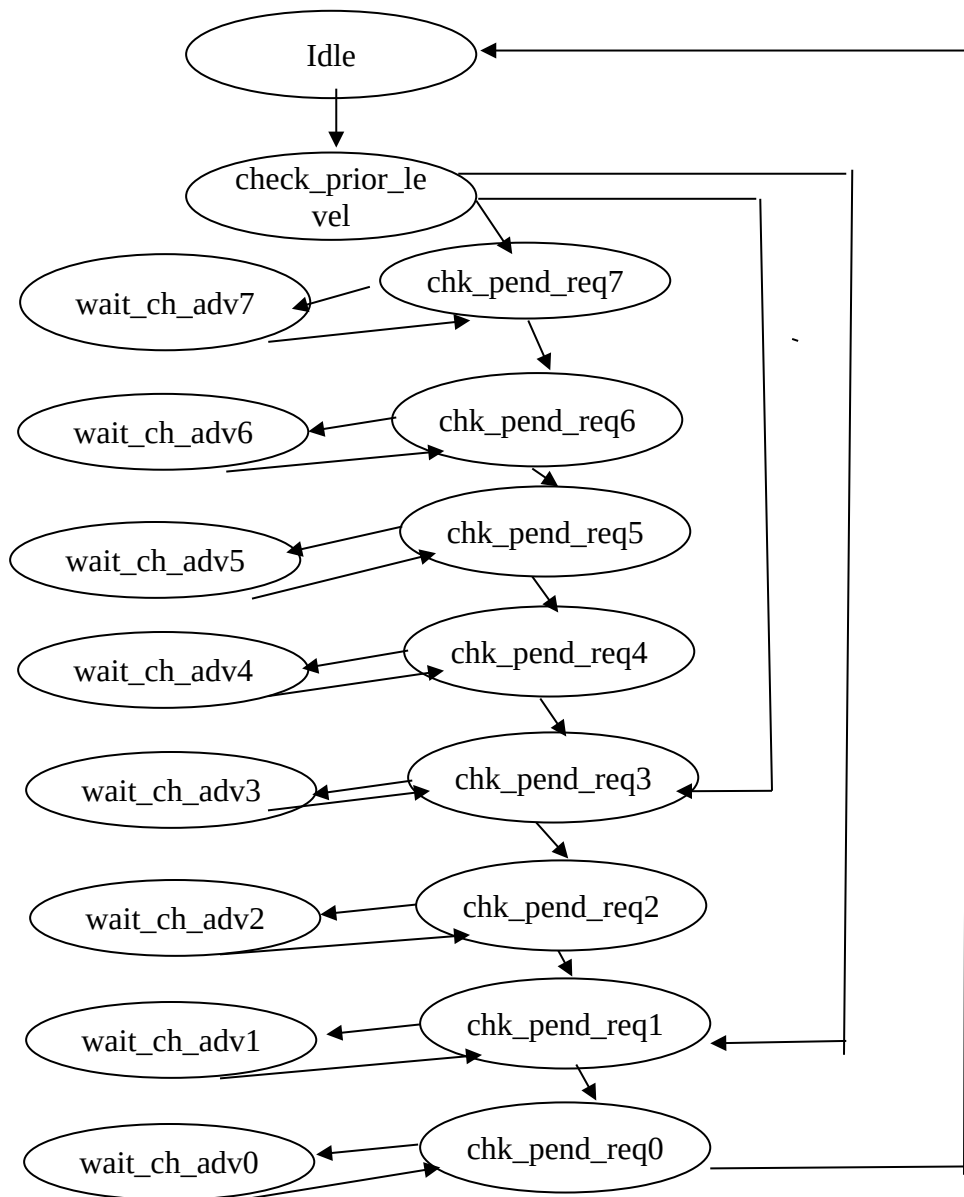


Fig 6: State diagram showing the priority arbiter

State description:

Idle : On reset this state is reached. When there is start arbitration signal from priority arbiter. The arbitration begins.

Idle -> **chk_prior_level** : if start arbitrate is asserted

chk_prior_level : The comparison of priority level is done here.

chk_prior_level -> **chk_pend_req7** : If the priority level is 8

chk_prior_level -> **chk_pend_req3** : If the priority level is 4

chk_prior_level -> **chk_pend_req1** : If the priority level is 2

chk_pend_req7 : Check for the pending request from priority fifo 7. If it is there, all the channels in the fifo are read until the pending request goes low.

chk_pend_req7 -> **wait_ch_adv7** : If there is pending request from priority fifo 7 assert the channel select and pass on the channel number to mux.

chk_pend_req7 -> **chk_pend_req6** : when all the pending request signal is zero.

wait_ch_adv7: Wait until there is channel advance signal from dma.

wait_ch_adv7 -> **chk_pend_req6** : When channel _advance is asserted.

chk_pend_req6 : Check for the pending request from priority fifo 6. If it is there, all the channels in the fifo are read until the pending request goes low.

chk_pend_req6 -> **wait_ch_adv6** : If there is pending request from priority fifo 6 assert the channel select and pass on the channel number to mux.

chk_pend_req6 -> **chk_pend_req5** : when all the pending request signal is zero.

wait_ch_adv6: Wait until there is channel advance signal from dma.

wait_ch_adv6 -> **chk_pend_req5** : When channel _advance is asserted

chk_pend_req5 : Check for the pending request from priority fifo 5. If it is there, all the channels in the fifo are read until the pending request goes low.

chk_pend_req5 -> **wait_ch_adv5** : If there is pending request from priority fifo 5 assert the channel select and pass on the channel number to mux.

chk_pend_req5 -> **chk_pend_req4** : when all the pending request signal is zero.

wait_ch_adv5: Wait until there is channel advance signal from dma.

wait_ch_adv5 -> **chk_pend_req4** : When channel _advance is asserted

chk_pend_req4 : Check for the pending request from priority fifo 4. If it is there, all the channels in the fifo are read until the pending request goes low.

chk_pend_req4 -> wait_ch_adv3 : If there is pending request from priority fifo 4 assert the channel select and pass on the channel number to mux.

chk_pend_req4 -> chk_pend_req3 : when all the pending request signal is zero.

wait_ch_adv4: Wait until there is channel advance signal from dma.

wait_ch_adv4 -> chk_pend_req4 : When channel _advance is asserted

chk_pend_req3 : Check for the pending request from priority fifo 3. If it is there, all the channels in the fifo are read until the pending request goes low.

chk_pend_req3 -> wait_ch_adv3 : If there is pending request from priority fifo 3 assert the channel select and pass on the channel number to mux.

chk_pend_req3 -> chk_pend_req2 : when all the pending request signal is zero.

wait_ch_adv3: Wait until there is channel advance signal from dma.

wait_ch_adv3 -> chk_pend_req3 : When channel _advance is asserted

chk_pend_req2 : Check for the pending request from priority fifo 2. If it is there, all the channels in the fifo are read until the pending request goes low.

chk_pend_req2 -> wait_ch_adv2 : If there is pending request from priority fifo 2 assert the channel select and pass on the channel number to mux.

chk_pend_req2 -> chk_pend_req1 : when all the pending request signal is zero.

wait_ch_adv2: Wait until there is channel advance signal from dma.

wait_ch_adv2 -> chk_pend_req2 : When channel _advance is asserted

chk_pend_req1 : Check for the pending request from priority fifo 1. If it is there, all the channels in the fifo are read until the pending request goes low.

chk_pend_req1 -> wait_ch_adv1 : If there is pending request from priority fifo 1 assert the channel select and pass on the channel number to mux.

chk_pend_req1 -> chk_pend_req0 : when all the pending request signal is zero.

wait_ch_adv1: Wait until there is channel advance signal from dma.

wait_ch_adv1 -> chk_pend_req1 : When channel _advance is asserted

chk_pend_req0 : Check for the pending request from priority fifo 0. If it is there, all the channels in the fifo are read until the pending request goes low.

chk_pend_req0 -> wait_ch_adv6 : If there is pending request from priority fifo 0 assert the channel select and pass on the channel number to mux.

chk_pend_req0 -> Idle : when all the pending request signal is zero.

wait_ch_adv0: Wait until there is channel advance signal from dma.

wait_ch_adv0 -> chk_pend_req0 : When channel _advance is asserted

3.1.3 Multiplexer

All the select lines will come from arbiter to multiplexer. Depending on the channel selected, all the parameters required for that channel are given to DMA engine. Description of MUX is skipped here, as it is self-explanatory.

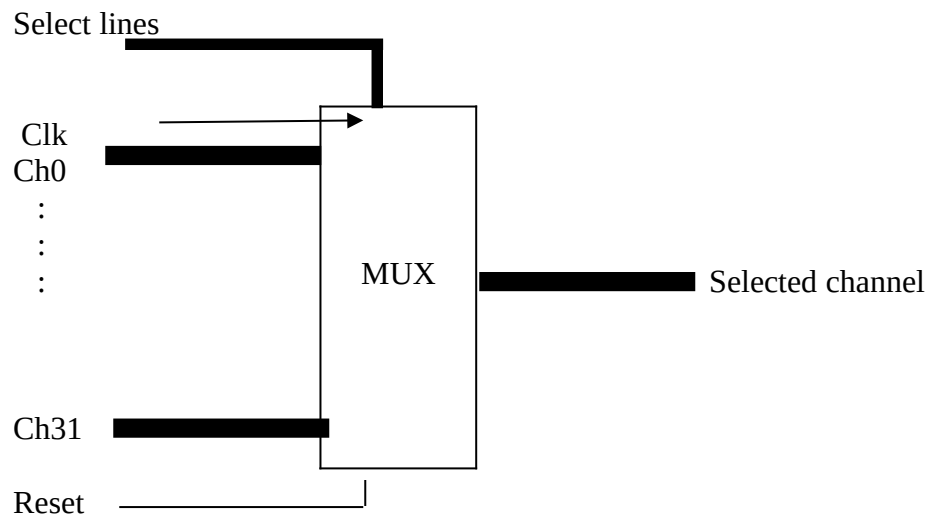
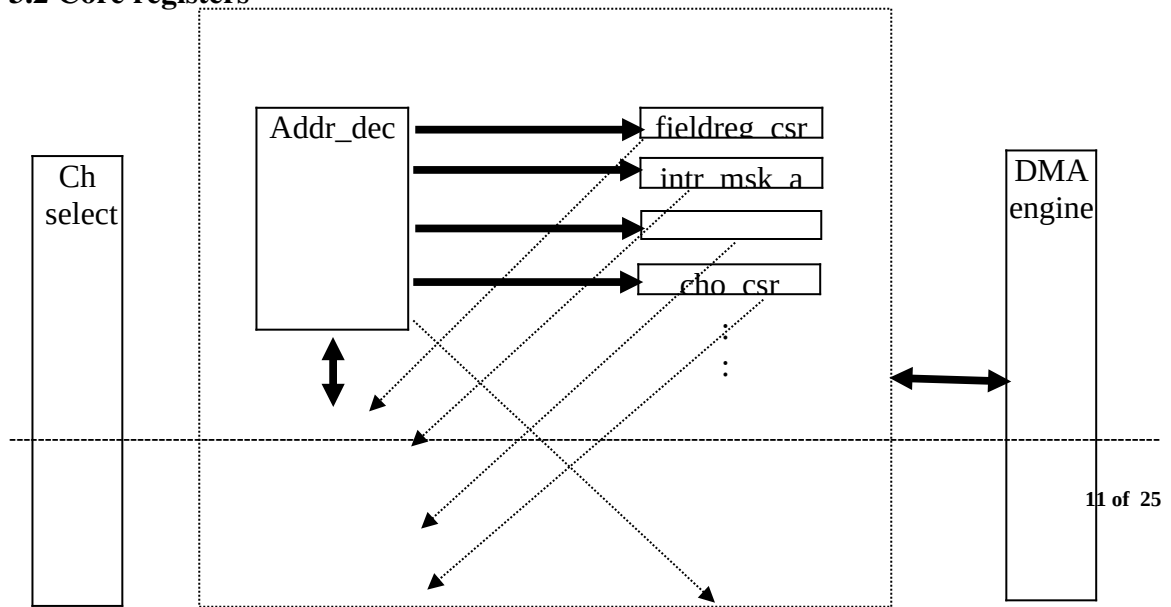


Fig 7 Multiplexer

3.2 Core registers



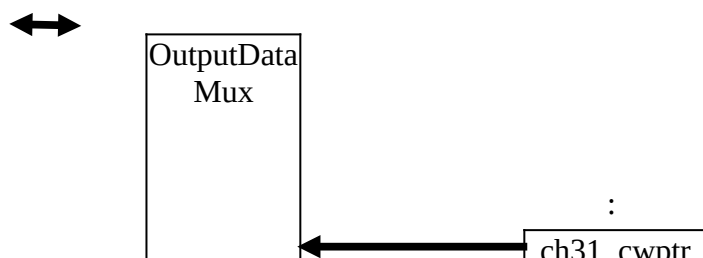


Fig 8 Block diagram of core register

Core register set contains various blocks like address decoder, output data multiplexer and each register have one separate instance of filed register module.

Address decoder module will decode the address of the registers and enables a select signal to select the register. Output data multiplexer module will get the select signal from address decoder module and if read is asserted then, the multiplexer will output the data from the selected register.

Each register has instance of field register module. The parameters for each register differ. The parameters are value after reset, location of reserved bits, read and write bits, read only bits and read write and cleared bits. All the parameters are 32 bits wide. The address and accessibility of different bits in registers is given in reference document in the introduction.

Interrupt is generated in the register set module. If any of the interrupt is set and mask bit is also set on that particular field then the interrupt is generated.

3.3 DMA Engine

DMA engine has three modes of transfers.

- Normal mode
- HW handshake mode and
- Linked list descriptors

Normal mode :

In this mode, DMA performs block copy from one location to another, either on the same interface or on a different interface. When the start request from channel select module is asserted, the dma engine begins transfer.

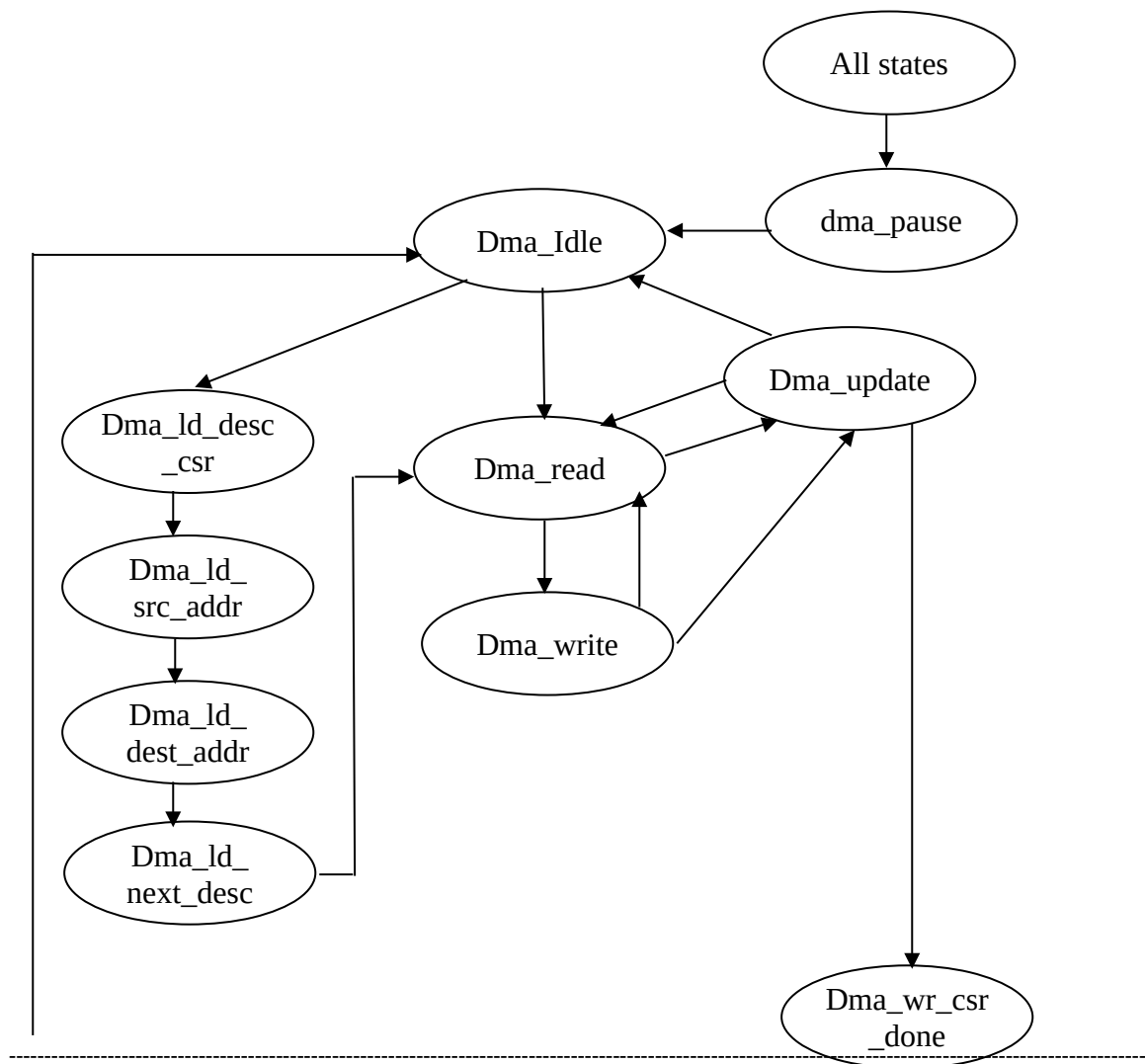
HW handshake mode :

In this mode one full transfer requires more than one external trigger. DMA_REQ has to be asserted for starting DMA transfer.

Linked list descriptors :

In this mode DMA engine will fetch the channel descriptors from memory attached to interface 0. The descriptors are provided in a linked list format. After completion new

descriptor may be loaded. The address of valid descriptor is programmed in linked list descriptor pointer register. The chunk size is also set in channel SZ register. The USE_ED bit in the channel CSR register must be set to enable this transfer.



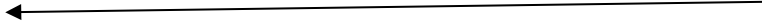


Fig 9 State machine for DMA engine

State machine

dma_Idle : On reset this state is reached. All the variables are assigned their initial values in this state.

dma_idle -> dma_pause : When pause request is asserted all the states should move to dma_pause state.

dma_idle -> dma_read : when dma_start and ch_csr_reg[USE_ED] bit is zero then, the state transition happens.

dma_idle -> dma_ld_desc_csr : When dma_start and ch_csr_reg[USE_ED] bit is one then, the state transition occurs.

dma_pause : All the states should have access to this state. When csr_reg[PAUSE] is asserted all the states should come to this state after the current transfer.

dma_pause -> dma_idle : When csr_reg[PAUSE] is de-asserted, the state machine should start and go back to idle.

dma_update : During read or write if ch_csr_reg[STOP] bit is set, then the states transit to this. If the transfer is complete then also all states come to this to update registers. In this state, the error is asserted and goes back to idle state if ch_csr_reg[STOP] bit is set. If stop is not set, the state will update the required registers.

In this state, the check has to be done to see if ch_csr_reg[ARS] bit is set and if so, then the ch_csr_reg[REST] is also set then, the state should go to read state.

dma_update -> dma_idle : State transition occurs when not descriptor transfer. If it is a descriptor transfer, and not end of list bit asserted. On error, assert ch_csr_reg[ERR] bit and go back to idle.

dma_update -> dma_wr_csr_done : When desc_reg[EOL] bit is set, the control should flow like this.

dma_update -> dma_read : when ch_csr_reg[ARS] == 1 && ch_csr_reg[REST] == 1 then, this should happen to restart the dma transfer.

dma_read : For read access this state is reached.

dma_read -> dma_write : When master_drdy is asserted, the transfer goes to write.

dma_read -> dma_update : when ch_csr_reg[STOP] bit is set, the DMA transfer is aborted and error signal is asserted.

dma_write : For write access this state is reached.

dma_write -> dma_update : When ch_csr_reg[STOP] bit is set or when chunk transfer is complete.

dma_write -> dma_read : When chunk of transfer is not complete, then it goes back to read state.

Note : descriptor_csr is a temporary register in module to store the descriptor data.

dma_ld_desc_csr : When ch_csr_reg[USE_ED] is set, then from idle state the transfer happens updating ch_csr register. ch_csr_reg[4:1] = descriptor_csr[19:16]

ch_csr_reg[11:0] = descriptor_csr[11:0]

dma_ld_desc_csr -> dma_ld_src_addr : When master_drdy is set.

dma_ld_src_addr : Writing of source address will happen here.

ch_csr_src_addr = descriptor_src_addr

dma_ld_src_addr -> dma_ld_dest_addr : When master_drdy is set.

dma_ld_dest_addr : Writing of destination address will happen here.

ch_csr_dest_addr = descriptor_dest_addr

dma_ld_dest_addr -> dma_ld_next_addr : When master_drdy is set.

dma_ld_next_addr : Writing of next descriptor address will happen here.

ch_desc = descriptor_next_addr

dma_ld_next_addr -> dma_read : When Master drdy is asserted.

dma_wr_csr_done : writes the ch_csr_reg[DONE] bit and goes back to idle.

dma_wr_csr_done -> dma_idle : next cycle.

Table 6 in appendix, shows the control signal flow in dma engine.

The ch_csr_src_sel and ch_csr_dest_sel bits indicate which master and which slave are accessed. Then the transfer of data happens as directed.

Address incrementing in the dma engine is done using circular buffers fashion. Circular buffer is the one in which the buffers never go beyond allocated memory space. These buffers will wrap around when last entry in the buffer is reached. Size of address mask register is 28 bits. The upper 28 bits in address location are incremented only if the mask bit for that particular bit is set.

Address increment is done using another counter module in which the 15 bits data is incremented in first phase then, the rest 15 bits are added to it to improve the performance of a counter.

Once the Address is incremented the next address [31:4] is the value of the incremented address [31:4] only if mask [31:4] bits are asserted. Each bit is compared separately.

If the ch_sft_ptr[swptr_en] is set then, the DMA engine has to always compare the software pointer value with the incremented address. If both are equal or if incremented value is more than the one in software pointer register, then, the write operation will halt until software pointer is updated. Software should take care of this updating.

The DMA done signal will be asserted once the chunk of data is transmitted. If total size of the transfer is over then, dma all done signal will be asserted. For all this assertion a counter is required to count the data that's been transferred. After every valid transfer the counter should be incremented to check the number of words transferred. If the transfer reaches the chunk size then, the dma done signal is asserted. If the transfer reaches the total size then, the dma all done signal is asserted.

ch_csr_reg[REST] bit is enabled along with ch_csr_reg[ARS], then in the update state the check has to be done and the flow should go to read state until the bit is cleared.

3.4 Wishbone interface

Each Wishbone interface has one master and one slave module.

3.4.1 Wishbone master

When passthrough logic is enabled, the combinational circuit comes into picture and passes all the signals to slave.

3.4.2 Wishbone slave

When passthrough logic is enabled, the combinatorial circuit will pass the signals to master.

3.4.3 Handshaking Protocol

All WISHBONE bus cycles use a handshaking protocol between the MASTER and SLAVE interfaces. The MASTER asserts o_mast_strobe when it is ready to transfer data. o_mast_strobe remains asserted until the SLAVE asserts one of the cycle terminating signals i_mast_ack, i_mast_err or i_mast_retry. At every rising edge of clock the terminating signal is sampled. If it is asserted, then o_mast_strobe is negated. Both sides of the interface can then completely control the rate at which data is transferred. If the SLAVE always operates at the maximum speed of the core, and if the i_mast_err and i_mast_retry signals are not used, then the o_slv_ack signal may be asserted by the slave immediately after it receives i_slv_strobe. The interface will function normally under these circumstances.

The use of o_slv_ack will terminate a local bus cycle. However, the SLAVE can optionally terminate the cycle with an error o_slv_err, or request that the cycle be retried with o_slv_retry.

All interfaces include the o_slv_ack terminator signal. Asserting this signal during a bus cycle causes it to terminate normally.

Asserting the optional `o_slv_err` signal during a bus cycle will terminate the cycle. It also serves to notify the MASTER that an error occurred during the cycle. This signal is generally used if an error was detected by SLAVE logic circuitry. For example, if the SLAVE has a parity-protected memory, then its `o_slv_err` signal can be asserted if a parity fault is detected.

Asserting the optional `o_slv_retry` signal will terminate a bus cycle. It also serves to notify the MASTER that the current cycle should be aborted, and retried at a later time. This signal is generally used for shared memory and bus bridges. In these cases, SLAVE circuitry would assert `o_slv_retry` if the local resource is busy.

3.4.4 Single READ / WRITE Cycles

The single READ / WRITE cycles are the basic modes of data transfer. The MASTER begins the cycle at the rising edge of clock edge 0. At that time it places an address onto address bus `o_mast_addr`, negates the `o_mast_we` line to indicate a READ cycle, drives the data select line(s) `o_mast_byte_sel` high or low depending upon the location that is being read, and asserts `o_mast_strobe` and `o_mast_cyc` to indicate the start of a new bus cycle.

The interconnection logic decodes the address and responds by placing asserting `i_slv_strobe` to the SLAVE. The SLAVE then places data on its data output bus `o_slv_data`. The SLAVE then terminates the cycle by asserting `o_slv_ack`. This informs the MASTER that it should latch the data on the next rising edge of clock. After latching the data, the MASTER terminates the cycle by negating `o_mast_strobe`.

The WRITE cycle is similar to the READ cycle. However, in this case the MASTER asserts `o_mast_we` and presents valid data out on `o_mast_data` at the beginning of the cycle. In response, the SLAVE asserts `o_slv_ack` when it is ready to latch the data at the next rising edge of clock.

The `o_mast_cyc` signal is mainly used for multi MASTER systems. This signal is asserted for the duration of all bus cycles.

4 Parameters to be measured

After designing the entire module, these fields needed to be filled for comparison.

Fields	Verilog	System verilog 3.0	System verilog 3.1
Total lines			
Compile time			
Run time			
Compile time			
Memory usage			

5 Appendix

Interface signals to different modules:

Signal name	Width	Description
<i>Inputs</i>		
i_clk	1	Global Clock
i_reset	1	Global Reset
i_next_ch	1	From DMA engine, indicating that DMA of current channel is done and next channel can be selected
i_dma_ack	1	Acknowledge from DMA indicating that either chunk transfer or total transfer is done
i_req	32	Input from Host indicating to start the DMA hardware handshake mode
i_nd	1	Input from host indicating DMA engine to fetch the channel descriptors from the memory attached to interface 0
i_dma_busy	1	Indication from DMA engine that, the DMA is busy
i_ch0_csr	32	Channel CSR value from regfile
i_ch0_sz	32	Channel transfer size from regfile
i_ch0_addr0	32	Channel address0 from regfile
i_ch0_am0	32	Channel address0 mask from regfile
i_ch0_addr1	32	Channel address1 from regfile
i_ch0_am1	32	Channel address1 mask from regfile
i_ch0_desc	32	Channel linked list descriptor pointer from regfile
i_ch0_swptr	32	Channel software pointer from regfile
i_ch1_csr	32	Channel CSR value from regfile
i_ch1_sz	32	Channel transfer size from regfile
i_ch1_addr0	32	Channel address0 from regfile
i_ch1_am0	32	Channel address0 mask from regfile
i_ch1_addr1	32	Channel address1 from regfile
i_ch1_am1	32	Channel address1 mask from regfile
i_ch1_desc	32	Channel linked list descriptor pointer from regfile
i_ch1_swptr	32	Channel software pointer from regfile
..... to		
i_ch31_csr	32	Channel CSR value from regfile
i_ch31_sz	32	Channel transfer size from regfile
i_ch31_addr0	32	Channel address0 from regfile

Signal name	Width	Description
i_ch31_am0	32	Channel address0 mask from regfile
i_ch31_addr1	32	Channel address1 from regfile
i_ch31_am1	32	Channel address1 mask from regfile
i_ch31_desc	32	Channel linked list descriptor pointer from regfile
i_ch31_swptr	32	Channel software pointer from regfile
<i>Outputs</i>		
o_pointer	32	Linked list descriptor pointer
o_prev_pointer	32	Linked list previous pointer
o_csr	32	Configuration and status register value from regfile
o_tx_size	32	Channel transfer size
o_src_addr	32	Source address
o_dest_addr	32	Destination address
o_src_addr_mask	32	Source address mask
o_dest_addr_mask	32	Destination address mask
o_ch_sel	5	selected Channel value
o_ndnr	32	Next descriptor with no request
o_dma_start	1	To DMA engine requesting the start of DMA

Table 1 showing the input output signals of channel select

Signal Name	Width	Description
<i>Inputs</i>		
i_clk	1	Global Clock
i_reset	1	Global reset
i_ch_valid	32	Channel valid from regfile
i_ch_req	1	Request from host
i_ch0_prior	3	Channel priority from regfile
i_ch1_prior	3	Channel priority from regfile
i_ch2_prior	3	Channel priority from regfile
i_ch3_prior	3	Channel priority from regfile
i_ch4_prior	3	Channel priority from regfile
i_ch5_prior	3	Channel priority from regfile
i_ch6_prior	3	Channel priority from regfile
i_ch7_prior	3	Channel priority from regfile
i_ch8_prior	3	Channel priority from regfile
i_ch9_prior	3	Channel priority from regfile
i_ch10_prior	3	Channel priority from regfile
i_ch11_prior	3	Channel priority from regfile
i_ch12_prior	3	Channel priority from regfile
i_ch13_prior	3	Channel priority from regfile
i_ch14_prior	3	Channel priority from regfile
i_ch15_prior	3	Channel priority from regfile
i_ch16_prior	3	Channel priority from regfile
i_ch17_prior	3	Channel priority from regfile

Signal Name	Width	Description
i_ch18_prior	3	Channel priority from regfile
i_ch19_prior	3	Channel priority from regfile
i_ch20_prior	3	Channel priority from regfile
i_ch21_prior	3	Channel priority from regfile
i_ch22_prior	3	Channel priority from regfile
i_ch23_prior	3	Channel priority from regfile
i_ch24_prior	3	Channel priority from regfile
i_ch25_prior	3	Channel priority from regfile
i_ch26_prior	3	Channel priority from regfile
i_ch27_prior	3	Channel priority from regfile
i_ch28_prior	3	Channel priority from regfile
i_ch29_prior	3	Channel priority from regfile
i_ch30_prior	3	Channel priority from regfile
i_ch31_prior	3	Channel priority from regfile
i_rd_fifo7	1	Read request from arbiter to read prior fifo
i_rd_fifo6	1	Read request from arbiter to read prior fifo
i_rd_fifo5	1	Read request from arbiter to read prior fifo
i_rd_fifo4	1	Read request from arbiter to read prior fifo
i_rd_fifo3	1	Read request from arbiter to read prior fifo
i_rd_fifo2	1	Read request from arbiter to read prior fifo
i_rd_fifo1	1	Read request from arbiter to read prior fifo
i_rd_fifo0	1	Read request from arbiter to read prior fifo
<i>Outputs</i>		
o_start_arbiter	1	Output to arbiter to begin the arbitration
o_rd_rdy	1	Output to arbiter indicating read request is ready
o_ch_num7	5	Output to arbiter passing the channel number in fifo
o_ch_num6	5	Output to arbiter passing the channel number in fifo
o_ch_num5	5	Output to arbiter passing the channel number in fifo
o_ch_num4	5	Output to arbiter passing the channel number in fifo
o_ch_num3	5	Output to arbiter passing the channel number in fifo
o_ch_num2	5	Output to arbiter passing the channel number in fifo
o_ch_num1	5	Output to arbiter passing the channel number in fifo
o_ch_num0	5	Output to arbiter passing the channel number in fifo
o_pend_req7	1	Output to arbiter indicating that request to be served
o_pend_req6	1	Output to arbiter indicating that request to be served
o_pend_req5	1	Output to arbiter indicating that request to be served
o_pend_req4	1	Output to arbiter indicating that request to be served
o_pend_req3	1	Output to arbiter indicating that request to be served
o_pend_req2	1	Output to arbiter indicating that request to be served
o_pend_req1	1	Output to arbiter indicating that request to be served
o_pend_req0	1	Output to arbiter indicating that request to be served

Table 2 showing the input output signals of priority encoder

Signal name	Width	Description
<i>Inputs</i>		
i_clk	1	Global clock
i_reset	1	Global reset
i_ch_valid	32	Channel valid signal from channels
i_start_arbitrate	1	Start arbitrate from priority encoder
i_pend_req7	1	Pending request in priority fifo
i_pend_req6	1	Pending request in priority fifo
i_pend_req5	1	Pending request in priority fifo
i_pend_req4	1	Pending request in priority fifo
i_pend_req3	1	Pending request in priority fifo
i_pend_req2	1	Pending request in priority fifo
i_pend_req1	1	Pending request in priority fifo
i_pend_req0	1	Pending request in priority fifo
i_ch_num7	5	Pending request channel number
i_ch_num6	5	Pending request channel number
i_ch_num5	5	Pending request channel number
i_ch_num4	5	Pending request channel number
i_ch_num3	5	Pending request channel number
i_ch_num2	5	Pending request channel number
i_ch_num1	5	Pending request channel number
i_ch_num0	5	Pending request channel number
i_rd_rdy	1	Read value is ready from priority encoder
i_channel_advance	1	From dma
<i>Outputs</i>		
o_ch_gnt	32	Channel grant to mux
o_gnt_ch_num	3	Channel number to mux
o_rd_fifo7	1	Read priority fifo
o_rd_fifo6	1	Read priority fifo
o_rd_fifo5	1	Read priority fifo
o_rd_fifo4	1	Read priority fifo
o_rd_fifo3	1	Read priority fifo
o_rd_fifo2	1	Read priority fifo
o_rd_fifo1	1	Read priority fifo
o_rd_fifo0	1	Read priority fifo

Table 3 showing the input output interface of channel arbiter

Signal name	Width	Description
<i>Inputs</i>		
i_clk	1	Global clock
i_reset	1	Global reset
i_wr_data	32	Data input
i_addr	32	Address in
i_wr_n_rd	1	If asserted write else read

Signal name	Width	Description
<i>Outputs</i>		
o_intr	1	Interrupt indication
o_rd_data	32	Data after read
o_main_csr	32	Main csr register values
o_int_msk_a	32	Interrupt mask for INTA_O output
o_int_mask_b	32	Interrupt mask for INTB_O output
o_int_src_a	32	Interrupt source for INTA_O output
o_int_src_b	32	Interrupt source for INTB_O output
o_ch0_csr	32	CSR register
o_ch0_sz	32	Total size
o_ch0_addr0	32	address0
o_ch0_addr_mask0	32	Address mask 0
o_ch0_addr1	32	Address 1
o_ch0_addr_mask1	32	Address mask 1
o_ch0_desc_ptr	32	Linked list descriptor pointer
o_ch0_swptr	32	Software pointer
:		
:		
o_ch31_csr	32	CSR register
o_ch31_sz	32	Total size
o_ch31_addr0	32	address0
o_ch31_addr_mask0	32	Address mask 0
o_ch31_addr1	32	Address 1
o_ch31_addr_mask1	32	Address mask 1
o_ch31_desc_ptr	32	Linked list descriptor pointer
o_ch31_swptr	32	Software pointer

Table 4 showing the interface of core registers

Signal name	Width	Description
<i>Inputs</i>		
i_clk	1	Global Clock
i_reset	1	Global Reset
i_dma_start	1	From channel select indicating the DMA to start
i_dma_nd	1	Next descriptor
i_req	1	Input from Host indicating to start the DMA hardware handshake mode
i_ch_csr	32	Channel CSR value from regfile
i_ch_sz	32	Channel transfer size from regfile
i_ch_addr0	32	Channel address0 from regfile
i_ch_am0	32	Channel address0 mask from regfile
i_ch_addr1	32	Channel address1 from regfile
i_ch_am1	32	Channel address1 mask from regfile
i_ch_desc	32	Channel linked list descriptor pointer from regfile
i_ch_swptr	32	Channel software pointer from regfile

Signal name	Width	Description
i_dma_stop	1	From regfile ch_csr_reg[STOP]
i_dma_pause	1	From regfile csr_reg[PAUSE]
i_mast0_data	32	Master0 data
i_mast0_err	1	Master error bit
i_mast0_drdy	1	Master0 data ready signal
i_mast1_data	32	Master1 data
i_mast1_err	1	Master1 error bit
i_mast1_drdy	1	Master1 data ready signal
<i>Outputs</i>		
o_ch_csr	32	Channel CSR value from regfile
o_ch_addr0	32	Channel address0 to regfile
o_ch_addr1	32	Channel address1 to regfile
o_ch_next_desc	32	Channel linked list descriptor next pointer to regfile
o_ch_csr_we	1	Channel CSR value to regfile
o_ch_addr0_we	1	Channel address0 to regfile
o_ch_addr1_we	1	Channel address1 to regfile
o_ch_next_desc_we	1	Channel linked list descriptor pointer to regfile
o_status_fetch_desc	1	Status indicating the fetching descriptor
o_mast0_start	1	Start master0
o_mast0_wait	1	Insert wait states
o_mast0_wr_n_rd	1	If set write else read
o_mast0_data	32	Master data out
o_mast0_adddr	32	Master address
o_mast1_start	1	Start master1
o_mast1_wait	1	Insert wait states
o_mast1_wr_n_rd	1	If set write else read
o_mast1_data	32	Master data out
o_mast1_adddr	32	Master address
o_dma_ack	1	Acknowledge to dma
o_dma_nxt_desc	1	Next descriptor
o_dma_bsy	1	Status dma busy
o_dma_done	1	Dma chunk transfer done
o_dma_done_all	1	Complete Dma transfer is over
o_dma_err	1	Error status when ch_csr_reg[STOP] is asserted

Table 5 showing the DMA input output signals

Signal	States									
	Idle	Read	Write	Update	Pause	ld_csr	ld_ad dr0	ld_ad dr1	ld_nd	Wr_d one
o_ch_csr	x	x	x	x	x	csr	x	x	x	X
o_ch_addr0	x	x	x	x	x	x	addr	x	x	X
o_ch_addr1	x	x	x	x	x	x	x	addr	x	x
o_ch_next_desc	x	x	x	x	x	x	x	x	nd	x
o_ch_csr_we	0	0	0	0	0	1	0	0	0	0

Signal	States									
o_ch_addr0_we	0	0	0	0	0	0	1	0	0	0
o_ch_addr1_we	0	0	0	0	0	0	0	1	0	0
o_ch_next_desc_we	0	0	0	0	0	0	0	0	1	0
o_status_fetch_desc	0	0	0	0	0	1	1	1	1	0
o_mast_start	0	1	1	0	0	0	0	0	0	0
o_mast_wr_n_rd	0	0	1	0	0	0	0	0	0	0
o_dma_ack	0	0	0	1	0	0	0	0	0	0
o_dma_nxt_desc	0	0	0	1	0	0	0	0	0	0
o_dma_bsy	0	1	1	1	1	1	1	1	1	1
o_dma_done	0	0	0	1	0	0	0	0	0	0
o_dma_done_all	0	0	0	1/0	0	0	0	0	0	0

Table 6 showing the control signals for dma engine

Signal name	Width	Description
Inputs		
i_clk	1	Global Clock
i_reset	1	Global Reset
i_dma_start	1	From channel select indicating the DMA to start
Inputs		
i_dma_req	1	Request from DMA
i_dma_nd	1	Indicating to force next descriptor
i_dma_rest	1	Indicating to restart the current transfer
i_mast_data	32	Data to Master
i_mast_ack	1	Ack to master indicating normal cycle termination
i_mast_err	1	To master indicating abnormal cycle termination
i_mast_retry	1	To master indicating master to retry this operation
i_slv_addr	32	Address input for slave
i_slv_data	32	Data to slave
i_byte_slv_sel	4	Bytes select for slave
i_slv_cyc	1	Encapsulate the valid transfer cycle to slave
i_slv_strobe	1	To slave indicating the valid transfer
Outputs		
o_mast_data	32	Data from master
o_mast_addr	32	Address from master
o_slv_data	32	Data from slave
o_mast_we	1	Write enable from master
o_mast_cyc	1	Encapsulate the valid transfer cycle from master
o_mast_strobe	1	From master indicating the valid transfer
o_slv_ack	1	Ack from slave indicating normal cycle termination
o_slv_err	1	From slave indicating abnormal cycle termination
o_slv_retry	1	From Slave indicating master to retry this operation
o_dma_ack	1	DMA acknowledge

Signal name	Width	Description
o_inta	1	Interrupt for Interface A
o_intb	1	Interrupt for Interface B

Table 7 showing the input output ports on WB interface